DressMe — Module M7 "Local catalogue"

Technical report · BO6 Tunisian market · ESPRIT 2026 Data collection (scraping), cleaning, a clothing search engine over Tunisian shops, and its evaluation.

Updated 22/10/2026: Qwen 2.5 7B LLM (§5.13, §6.6). 16/10/2026: 43 shops, 6,375 products. First version 14/10/2026 (15 shops, 2,471 products).

Contents

1. Context and goal
2. Overview
3. Data collection: scraping
4. Data cleaning
5. The M7 module
6. Evaluation
7. How to test the module
8. Limits and next steps
9. Appendix: module files

---

1. Context and goal

DressMe is a personal fashion assistant. Module **M7 "Local catalogue"** answers a specific need of the Tunisian market: a user looks for, say, *white sneakers in Tunis for a budget of 80 TND*. Today she has to browse dozens of Instagram pages and websites. M7 directly suggests pieces from Tunisian shops that fit her request, her budget and her city, whether she writes in French, English, Tunisian darija or Arabic.

Example result returned by the module:

> **Manuella · Tunis · Basket à plateforme femme · 69.3 TND · link to the product page**

Place in the project. According to the *Task Split* document, M7 is a **shared module**: built once by BO6 and called by other BOs (BO1, BO2, BO3, BO5, BO7, BO8, BO9). It must therefore expose a stable API. The architecture document (*DressMe_MultiAgent_Pipeline.pdf*, section 7 "Local shop agent") sets:

Item	Architecture requirement	What was built
Knowledge base	CSV/JSON of shops and pieces, no live scraping	Cleaned CSV of 6,375 products (43 shops) + image index, in `ml/data/shops/`
Search	Embeddings + BM25 (hybrid)	FashionCLIP (image + text) + BM25
Filters	Price (TND), city, category, style	Price, city/governorate, category, **colour**, **gender**, stock, style vector

Item	Architecture requirement	What was built
Integration	Tool callable by the orchestrator (LangGraph)	`localshop` tool + REST API `/m07/*` + BO6 Flutter screen
Input / output	`query, gap, style_vector` → `shop, area, item, price_tnd, match`	Same fields, enriched (links, photo, gender, reasons, pagination)

The additional request was to run **FashionCLIP locally in Python**, which was done (§5.2). This choice was then compared with two more recent models on our catalogue (§5.12), and the module was **evaluated** on 72 hand-written requests (§6).

---

2. Overview

The work has two parts: **offline** data preparation (re-run from time to time to refresh prices and stock), then a very fast **online** search.

```
OFFLINE (preparation)                      ONLINE (each search, ~40 ms)

 Shop websites (43)                         "baskets blanches à Tunis 80 TND"
    | 1. scraper/run.py                              |
    ▼                                               ▼
 local_catalogue.csv + images/ (6,760)      parse    -> sneakers, white, Tunis, ≤ 80 TND
    | 2. scraper/clean.py                            |    (darija and Arabic translated first)
    ▼                                               ▼
 local_catalogue_clean.csv (6,375)          retrieve -> FashionCLIP + BM25
    | 3. pipeline/ingest.py                          |
    ▼                                               ▼
 index/ (1 vector per photo)  ──────────────► filter   -> price, city, type, colour, gender
    | 4. pipeline/attributes.py                      |
    ▼                                               ▼
 gender estimated from the photo            rerank   -> score, style, diversity, reasons
```

All the data (CSV, photos, index, reports) is in `ml/data/shops/` and is **tracked in git** on the `BO6/local_catalogue` branch (≈ 340 MB, of which 327 MB of photos), so the team gets it with a plain `git pull`. The project README plans DVC for data: moving to DVC or Git LFS remains possible if the repository gets too heavy (§8).

2.1 Input → model → output

Each step takes an *input*, transforms it with a model or a method, and produces an *output* that becomes the input of the next step.

Step	Input	Model / method	Output
1. Scraping	A shop's data (e.g. Manuella's Shopify JSON)	Platform adapter + normalisation rules	1 raw CSV row + 1 JPEG photo

Step	Input	Model / method	Output
2. Cleaning	Raw row	Rules, *dHash* image fingerprint, HTTP check	1 clean row (28 columns)
3. Indexing	JPEG photo (800 px)	**FashionCLIP — image encoder** (pretrained neural network)	Vector of 512 numbers
4. Gender from the photo	Photo vector	**FashionCLIP zero-shot** ("women's ..." vs "men's ...")	`gender` + `gender_source = photo`, when the model is sure enough
5. Understanding	User's sentence (FR, EN, darija, Arabic)	Darija lexicon + `parse.py` rules	Criteria + English sentence
6. Query encoding	English sentence	**FashionCLIP — text encoder**	Vector of 512 numbers
7. Retrieval	Query vector + 6,375 photo vectors + words	Cosine similarity + BM25	2 scores per product
8. Filters and ranking	Scores + criteria	Filters, weighted sum, diversity (MMR)	The best k products, with `match` and reasons

Only steps 3, 4 and 6 use an artificial intelligence model (FashionCLIP). The others are rules and computations: fast, free and explainable.

Real end-to-end example

The product followed is **Manuella's "Basket à plateforme femme"** (women's platform sneaker); the request is the one of the brief.



Offline — the product enters the catalogue

Step 1 · Scraping. *Input*: the shop's public Shopify JSON (excerpt).

```
{"title": "Basket à plateforme femme", "vendor": "Manuella",
 "options": ["couleur", "Pointure"],
 "variants": [{"option1": "Blanc", "option2": "41", "price": "69.300"},
              {"option1": "Blanc", "option2": "40", "price": "69.300"}, "..."]}
```

Output: one raw row of `local_catalogue.csv` (and the photo `images/2b7db627f328cf6a.jpg`).

```
Manuella | Basket à plateforme femme | Sneakers | Blanc | 41, 40, 39, 38, 37, 36 | 69.3 TND
         | Tunis | true
```

Step 2 · Cleaning. *Input*: the raw row. *Output*: the clean row (excerpt). The gender is read from the name ("femme").

```
{"product_id": "prd_2b7db627f3", "category": "shoes", "subcategory": "sneakers",
 "label_fr": "Baskets", "color": "white", "color_fr": "Blanc", "color_hex": "#FFFFFF",
 "gender": "women", "gender_source": "text", "size": "36, 37, 38, 39, 40, 41",
 "size_type": "numeric", "price_tnd": 69.3, "currency": "TND", "city": "Tunis",
 "governorate": "Tunis", "product_url_status": "ok", "missing_fields": ""}
```

Step 3 · Indexing (FashionCLIP). *Input*: the photo above. *Output*: a vector of 512 numbers describing its visual content.

```
[-0.015, 0.046, -0.003, -0.019, -0.045, … 507 more numbers]
```

Online — the user's search

API *input* (`POST /m07/search`):

```
{"query": "baskets blanches à Tunis pour un budget de 80 TND", "k": 3}
```

Step 5 · Understanding. *Output*:

```
{"category": "shoes", "subcategory": "sneakers", "color": "white", "max_price_tnd": 80.0,
 "city": "Tunis", "governorate": "Tunis", "terms": ["basket", "blanche"],
 "prompt_en": "a photo of white sneakers", "translated": null, "corrected": null}
```

Step 6 · Query encoding (FashionCLIP). *Input*: `"a photo of white sneakers"`. *Output*: `[-0.047, 0.004, -0.097, -0.006, 0.003, … 507 more numbers]`.

Step 7 · Retrieval. For our sneaker: - cosine similarity between the query vector and the photo vector: **0.289** (the best score in the catalogue is 0.342); - BM25 score of the words "basket" and "blanche" in its record: **3.99**.

Step 8 · Filters and ranking. - Filters (sneakers, price ≤ 80 TND, Tunis governorate, **white**, in stock): **6,375 → 1 product**. Without the colour filter, 28 sneakers in 11 different colours would pass (§5.5). - Score components of our sneaker: - visual = (0.289 − 0.15) / (0.33 − 0.15) = **0.772**; - text = **1.0** (best candidate); - attributes (sneakers + white) = **1.0**; - no gender or style vector in this request. - `match` = (0.45 × 0.772 + 0.20 × 1.0 + 0.20 × 1.0) / (0.45 + 0.20 + 0.20) = **0.880**.

API *output* (excerpt):


```
{"shop": "Manuella", "area": "Tunis", "item": "Basket à plateforme femme", "price_tnd": 69.3,
 "match": 0.88, "product_id": "prd_2b7db627f3", "category": "shoes", "subcategory": "sneakers",
 "color": "white", "gender": "women", "size": "36, 37, 38, 39, 40, 41", "available": true,
 "image_path": "images/2b7db627f328cf6a.jpg",
 "product_url": "https://manuella.com.tn/products/basket-a-plateforme-femme",
 "reasons": ["Baskets", "Couleur : blanc", "69.3 TND (budget 80 TND)", "Boutique à Tunis",
             "Ressemble visuellement à la demande", "Marque tunisienne"]}
```


The reasons are in French: they are shown as they are to the user.

---

3. Data collection: scraping

3.1 Choice of sources

Data was collected in **two campaigns**.

Campaign 1 (14/10/2026): 37 sources, 15 shops kept. The starting point was a list of 28 sources: 5 marketplaces or directories (Ad. Mall, Moudda, Lofa, Hawwini, Boutique.tn), 7 ready-to-wear chains (VOG, HA, Apparel, Celina Fashion, MAGIC, UNJA, Corella) and 16 shops listed in the Hawwini directory. 9 sources were added for jewellery, accessories and shoes (Strass Accessoires, HK-STORE, GAŸA, The Queen Accessories, SKILA, Joliesse, Azar Shoes, Chaussures Pitillos, Shoes Center). Website addresses were found on Hawwini's retailer pages, then by web search. *Why:* guessing domains would have given wrong sites.

Campaign 2 (15/10/2026): city by city search, 28 shops added. Goal: cover more cities and the weak categories (traditional outfits, menswear, swimwear, lingerie, jewellery). Method: 1. web searches by city (Tunis, Ariana, Ben Arous, Sousse, Monastir, Mahdia, Sfax, Nabeul, Bizerte, Kairouan, Gabès, Djerba, Gafsa) and by category (leather shoes, bags, jewellery, jebba and kaftan, hijab and burkini, lingerie, menswear, straw bags); lists of Tunisian brands from *L'Instant M* and *Marhba*; 2. an **automatic probe** of each candidate: reachable? platform (Shopify, WooCommerce, PrestaShop, JSON-LD)? prices in TND? does `robots.txt` allow crawling? 3. exclusion of foreign chains (Celio, Kiabi), resellers of foreign labels (Le Sportif, Super Sport, DREST), second-hand (Sou9 Finds) and imported clothing (New Collection, Mahdia).

Sources left out, with the reason (also listed in `scraper/shops.py`, `SKIPPED`):

Source	Reason
ZEN, Souk El Kahina	Their `robots.txt` **disallows crawling**: respected
Mabrouk (Tunis, Nabeul, Sousse)	WooCommerce feed readable, but every photo ends in a redirect loop, then the site stops answering; to retry

Source	Reason
HKEYA (Ariana), Bassamet, HK-Store	Answer "429 Too Many Requests" even with 3 s between requests
Kastelo, El Khomssa	Custom sites, product pages not in the sitemap: needs a dedicated HTML parser
Barsha, Rosace, Fedia Fashion	JavaScript-only sites, or JSON-LD without name or photo
Diamelle	Fine jewellery and second-hand watches of foreign brands (Rolex…)
JENNAH Boutique, Ayada Boutique	**French** shops (prices in euros)
Scarpa Store, Sneakers Tunisie, Sneakers Outlet, Le Sportif, Super Sport, DREST	Resell **mostly foreign labels**
Celio, Kiabi, Chaussures Pitillos	Foreign chains or labels
Lofa	Demo catalogue (stock photos)
Celina Fashion, SKILA	Showcase sites without catalogue or prices
Dabchy, Sou9 Finds	Second-hand
Hawwini	Price comparator: only used to find addresses
GAŸA, The Queen Accessories, Azar Shoes	No website found
Modeco, Fripup, Spadri, Unill (Gafsa), Lascarfa (Sousse), Lunja, Angel (Mahdia), 100% Tounsi	Unreachable, down or invalid certificate
Ad. Mall, boutique.tn, D'Stock, Pink and Grey, Folla Création (Nabeul)	No structured data: needs a dedicated HTML parser

Result: 43 shops in the final catalogue.

3.2 Detecting each site's platform

Before writing any scraping code, each site was analysed: which e-commerce platform does it use? We look at the home page (`generator` tag, signatures such as `cdn.shopify.com` or `wp-content`) and test each platform's known data URLs. For campaign 2, this analysis was automated (probe of 48 sites).

Platform	Shops	How products are read
Shopify	Moudda, NOON, Manuella, Kontakt, SeaSide, Emraa, Gaya, Iley'com	Public feed `/products.json`
WooCommerce	VOG, Corella, Apparel, KAI Signature, Wivine, Lunar, AD Makeba, Opierre, Hraier, Caprice, Exclusive, Intimate, Lingerie.tn, Patrinova, Sougui	Public *Store API* `?rest_route=/wc/store/v1/products`
PrestaShop 1.7+	Exist, FEModest, Shoes Mall, Strass, Joliesse, Chill&Lit, Ultima, Baletto, Carolie, Cravate, CityMode, Hijabella, Lyséa, Ms. Marion, Maison ARA	Category pages requested as JSON (`X-Requested-With` header), sizes read on the product page
PrestaShop 1.6	LYOUM	HTML "product cards" (custom theme)
Pages with JSON-LD	MAGIC, UNJA, GET	`schema.org/Product` data on each product page (listed by the sitemap or category pages)

Platform	Shops	How products are read
Private API	HA	JSON API used by their own site

Why: platforms already expose their products as **structured data** (JSON). This is far more reliable than extracting text from HTML: price, currency, variants, stock and photos come already separated. We wrote **one adapter per platform** (`scraper/sources.py`) rather than code per site: the 28 shops of campaign 2 were each added with one line of configuration.

PrestaShop adapter fix. Baletto names its products by model ("Cinzia Noir Daim"): only the site's category says they are sandals. The adapter first visited the generic "femme" category and then ignored the same product in "sandales". It now **adds** every category met: Baletto goes from **2 to 162 products**.

3.3 Polite scraping

Rule	Implementation	Reason
Respect `robots.txt`	Every URL is checked before being requested; ZEN and Souk El Kahina, which disallow it, are not collected	Sites state what they allow
No overload	At least 0.5 s between two requests to one site (1 s for URL checks); a site answering 429 with 3 s between requests is dropped	Do not disturb the shops
Identifiable bot	User-Agent `DressMeCatalogueBot/0.1 (student project, ESPRIT)`	Transparency
Temporary errors	Up to 6 retries with growing waits (429, 5xx)	Robustness
No cookies	Stateless client	See below

Problem met: 429 errors on Shopify. Shopify blocks `/products.json` **as soon as its session cookies are present** (set while reading `robots.txt`). Solution: a client that refuses all cookies, which sites do not need to show a public catalogue.

3.4 Normalisation during collection

Each shop describes its products its own way. To get a consistent catalogue, keyword rules are applied (`scraper/normalize.py`):

- **Category** from the product name, then the shop's category. Rules cover French, English, Tunisian words and Arabic: *dengri* (jeans), *pul* (sweater), *joking* (jogging), *jebba*, *jelbeb*, *tabdila*, *جلباب*, *كسوة*... **Rule order matters**: "veste en jean" is a jacket, "short basket" is shorts (basketball), "haut de gamme" (premium) is **not** a top.
- **Shop spellings**: "Colier", "Gillet", and "Boucles d’oreilles" with a typographic apostrophe (’) are recognised.
- **Main colour**: colour named in the title, else the first colour variant. Codes like `T00849-NOIRDEL` are recognised.
- **Sizes**: only sizes **in stock** when the shop gives stock per size.
- **Prices**: only prices **in TND** are kept (currency checked for each platform).
- **City**: checked on the site or in a source (store pages, press), e.g. VOG → Bizerte, LYOUM → La Marsa, Carolie and SeaSide → Sfax, Ultima → Nabeul, Lyséa → Tunis, GET → Tunis, Sfax, Sousse, Gabès. **Left empty** when no source gives an address.

3.5 Filters applied during collection

Filter	Examples removed	Reason
Not fashion	Books, prayer beads, suitcases, cushions; **decor, furniture and food** of marketplaces (vases, mirrors, consoles, stools, cakes, sculptures…)	Not clothing
Children / babies	"Sandale fillette", "Baskets Garçonnet"	DressMe is about adult wardrobes
Foreign labels	Nike, Adidas, UGG, Moncler…; watches and jewellery (Bigotti, Rolex, Swarovski…); lingerie (Simone Pérèle, Triumph…)	BO6 goal: promote **local** brands and shops
No price or no photo	Kontakt publishes 237 of its 250 products without a photo	A product without a photo cannot be shown nor encoded

The non-fashion filter has **two levels**. Unambiguous words ("vase", "console", "gâteau") always exclude. Ambiguous words ("décoratif", "corbeille", "miroir", "porte-clés", "panier") exclude only when the name names **no piece**: "Sac avec charm décoratif", "Soutien-gorge corbeille" (balconette bra) and "Ceinture et porte-clés" stay; "Cœur décoratif" and "Miroir soleil en rotin" go. A first, stricter version wrongly removed 28 real pieces (bags with a decorative charm, balconette bras, boots with a decorative buckle…): they were found by comparing the removed products shop by shop, and a test now protects them.

Artisan marketplaces (Iley'com, Hraier, Patrinova, Sougui). They also sell decor and food, filed under misleading categories ("Haut-parleur", a speaker, filed with tops). For them (`name_only` in `shops.py`), a product is kept only when **its own name** names a piece of clothing or an accessory.

Catalogue balance. At most **300 products per shop**, picked in turn from each category, and the foreign-label filter is applied **before** this quota.

3.6 Product photos

One photo per product is downloaded into `ml/data/shops/images/`, converted to a JPEG of at most 800 px (**6,760 photos, 327 MB, about 45 KB each**). The file name comes from the product URL: it does not change between updates, and a photo already on disk is not downloaded again. *Why:* FashionCLIP needs the images locally, and the app shows them without depending on the shops' websites (route `/m07/images/…`).

3.7 Scraping result

Campaign 1 (15 shops):

Shop	Products read	Kept	Main exclusions
Moudda	1,250	300	272 foreign labels, 71 children
Manuella	700	300	73 children
VOG	1,200	300	—
Exist	1,163	300	12 not fashion
HA	791	300	29 children
Strass Accessoires	825	300	30 not fashion, 9 children

Shop	Products read	Kept	Main exclusions
Joliesse	357	300	18 not fashion (suitcases), 20 children
MAGIC, NOON, LYOUM, Apparel, Corella, UNJA, FEModest, Shoes Mall	581	433	books, home, children

Campaign 2 (28 shops):

Shop	Products read	Kept	Main exclusions
CityMode	1,200	300	**524 foreign labels**, 88 children
Iley'com	1,250	300	505 not fashion (decor)
Chill&Lit	1,246	300	237 not fashion, 161 children
Hraier	1,200	300	384 not fashion (pastries…), 241 without price
Kontakt	1,250	300	254 children (then 221 without photo)
Gaya	853	300	260 not fashion, 14 foreign labels
SeaSide, Exclusive, Intimate, Cravate	1,605	1,199	—
AD Makeba, GET, Baletto, Ms. Marion, Maison ARA	923	900	a few not fashion
Caprice, Lyséa, Carolie, Lingerie.tn, Patrinova, Opierre, Sougui, Lunar, Wivine, Ultima, Hijabella, KAI, Emraa	741	539	Sougui: 103 not fashion out of 129
Total (2 campaigns)		**6,760 with a photo**	256 products without a usable photo

3.8 Adding or refreshing shops

The `--merge` option scrapes only the given shops and keeps the others as they are, with their photos:

```
python -m modules.m07_local_catalogue.scraper.run --shops baletto --merge
# rate-sensitive site: 3 s between two requests
python -m modules.m07_local_catalogue.scraper.run --shops hkeya --merge --delay 3
```

Why: fixing Baletto only needed that shop (about 5 minutes) instead of re-running the 42 others.

4. Data cleaning

`scraper/clean.py` turns `local_catalogue.csv` (6,760 products) into `local_catalogue_clean.csv` (**6,375 products**). Every step is counted in `ml/data/shops/cleaning_report.md`. Category rules are re-applied to each product name: a rule fix improves the catalogue without scraping again.

Step	Products removed
Duplicates (63 within a shop, 32 across shops)	95
Not fashion (decor, furniture, food, key rings, suitcases…)	111
Marketplaces: name that names no piece	109
Foreign labels (model names sold without the brand)	58
Children	6
Dead product pages (404 or redirect to the home page)	6
Total	**385**

4.1 Prices as numbers, checked in TND

All prices are decimal numbers, strictly positive, in TND (guaranteed at the source by each adapter). Prices far from the median of their subcategory (less than a fifth or more than five times) are **flagged** in `quality_flags` (51 very low, 122 very high) but **never changed**. *Why:* an unusual price is not necessarily wrong (a 0.5 TND hijab pin is real). Two Hraier bags shown at **14,900 TND**, however, are very likely a typing error by the shop: they stay flagged and should be checked by hand (§8).

4.2 Final "local only" check

Cleaning re-runs the foreign-label filter with an extended list of model names sold without the brand in the title ("AIR FORCE 1 '07", "On Cloudtilt"…): **58 products removed**.

4.3 Common category naming

Categories follow the project's **shared data contract** (`GarmentCategory` : `top` , `bottom` , `dress` , `outerwear` , `shoes` , `accessory` , `traditional`), with a precise **subcategory** and a **French label**:

Before (shop text)	`category`	`subcategory`	`label_fr`
sneakers, baskets, basket, *sbadri*	shoes	sneakers	Baskets
chaussures, mocassins, derbies	shoes	shoes	Chaussures
costume, tailleur, smoking	dress	suit	Costumes
collier, colier, pendentif, chaîne	accessory	necklace	Colliers
bracelet, gourmette, manchette	accessory	bracelet	Bracelets
bague, alliance	accessory	ring	Bagues
boucles / puces d'oreilles	accessory	earrings	Boucles d'oreilles
jebba, kaftan, tabdila, جلباب	traditional	traditional	Tenues traditionnelles

Why: the Vision module (M1) classifies the wardrobe with the same categories, so M7 and M1 "speak the same language". Final split: accessories 1,706, shoes 1,655, tops 1,400, dresses / sets / suits / swimwear 695, bottoms 598, jackets and coats 242, traditional outfits 79.

4.4 Consistent colours, sizes, cities and gender

- **Colours**: an English code (`white` , `navy` ...) identical to the Vision module's, plus the French name (`Blanc`) and a hex code (`#FFFFFF`).
- **Sizes**: one notation, sorted (`2XL` → `XXL` , `ST` → `Taille unique`), and a `size_type` column (`letter` , `numeric` , `one_size` , `bra` ...).
- **Cities**: official name and **governorate** (`La Marsa` → `Tunis`). *Why:* a user searching "in Tunis" must also see the shops of La Marsa.
- **Gender** (new): `women` , `men` or `unisex` when the name or URL says it ("Basket femme", `/collections/homme`), with `gender_source = text` . When both genders or none are named, the gender stays empty; step 4 (§5.7) then fills it from the photo.

4.5 Duplicates

Two products are duplicates if they have **the same name, the same price and a near-identical photo**, compared with a **perceptual fingerprint** (64-bit *dHash*). Comparing only name and price would have been wrong: "Basket Homme" at 59.5 TND exists in 4 colours with 4 different photos. **95 duplicates removed** (63 within one shop, 32 across two shops).

4.6 URL checks

Every product page and photo URL was checked with an HTTP request (**13,202 URLs**). - A page answering 404/410, or redirecting to the home page, is **dead**: the product is removed (**6**). - Other failures are **flagged** without removing the product (38 pages still answering "429"). - Results are cached (`url_checks.json`) and **reused for 7 days**. *Why:* the first check took about 55 minutes; a re-run the next day (new rules) only re-checked the 302 failed URLs, in a few minutes, without hitting the 43 shops again.

4.7 Missing values marked, never invented

A missing value stays **empty** and the field name is listed in `missing_fields` .

Field	Filled	Why it is sometimes missing
Colour	85.5 %	Name without colour and no colour variant (jewellery, accessories...)
Size	64.5 %	Accessories and jewellery, or the shop does not publish sizes
City	43.1 %	Online shops without a published address (3,625 products)
Gender	92.0 %	Written in the name (1,834) or estimated from the photo (4,033); 508 unknown

Why: an invented colour or city could send the user to a shop that does not have the piece. The engine takes it into account (§5.5) and says so ("Couleur non précisée par la boutique", "Genre non précisé par la boutique").

4.8 Stable identifiers

Each product gets an id `prd_` + hash of its URL (e.g. `prd_2b7db627f3`). *Why:* other modules (favourites, partner links M13, sharing M10) need a durable reference, and the FashionCLIP index reuses vectors already computed thanks to it.

4.9 Final catalogue

Indicator	Value
Products	**6,375** (2,471 on 14/10)
Shops	**43** (15 on 14/10)
Tunisian brands / local multi-brand shops	3,627 / 2,748
In stock	5,393
Median price	78 TND
Gender	4,218 women, 1,641 men, 8 unisex, 508 unknown
Selection	771 bags, 606 lingerie pieces, 416 sandals, 403 sneakers, 291 shirts, 253 dresses, 243 necklaces, 174 heels, 111 rings, 79 traditional outfits, 40 black dresses (10 before)

Products per governorate (a chain counts in each city): Tunis 2,027, Sfax 1,450, Sousse 879, Nabeul 695, Bizerte 685, Ariana 631, Ben Arous 590, Monastir 590, Gabès 495, Médenine 296, Béja 90, Le Kef 90; 3,625 products from online shops without a city.

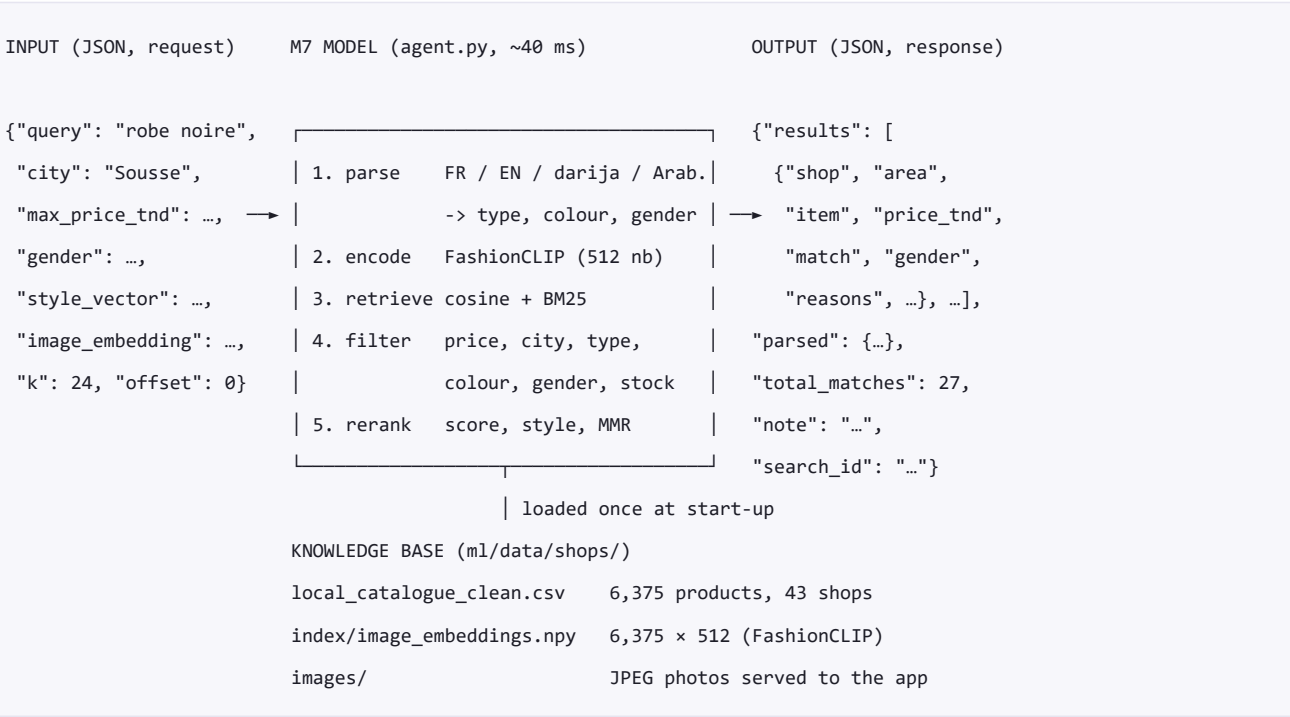
Shop	Products	Shop	Products	Shop	Products
Cravate	300	Hraier	220	Lyséa	79
Exclusive	300	GET	199	Kontakt	76
Manuella	300	Iley'com	181	Carolie	65
Intimate	299	Baletto	162	Lingerie.tn	49
SeaSide	299	Ms. Marion	151	Corella Store	48
HA	299	Maison ARA	112	Opierre	36
VOG	299	Patrinova	98	UNJA	36
Joliesse	296	MAGIC	90	FEModest	27
Strass Accessoires	294	NOON Clothing	90	Lunar	24
Exist	294	Caprice Lingerie	83	Wivine	23
Gaya	293	LYOUM	80	Apparel, Sougui	18
Chill&Lit	282			Ultima Boutique	15
Moudda	280			Hijabella	14
CityMode	274			KAI Signature	11
AD Makeba	247			Shoes Mall	9
				Emraa (ex-luma)	5

5. The M7 module

5.0 Input → model → output of M7

§2.1 follows the whole path, from scraping to search. Here we only look at **the M7 module itself**, that is what happens at each call of `POST /m07/search` (or of the `localshop` tool).

Two formats, two roles. The **input and output** are **JSON**: the format of the API messages, what the app, the assistant (M8) or another module sends to M7 and receives back. The **data** M7 searches is the **knowledge base**: the cleaned CSV, the FashionCLIP index and the photos. They are read **once**, when the API starts (`Catalogue.load`), then kept in memory; the CSV is never read again during a search, hence the ~40 ms per request.



Part	What it receives	What it does	What it produces
Input	A sentence (`query`), a wardrobe gap (`gap`), explicit filters (`city` , `max_price_tnd` , `subcategory` , `color` , `gender` …), optionally a style vector (M5) or image vector (M1), the page (`k` , `offset`)	Pydantic validation (`LocalShopInput`): at least one search criterion	A valid request, else a 422 error
Model	The request + the knowledge base in memory	Rules (understanding, filters) + **FashionCLIP** (the only learned part) + BM25 + weighted score	Ranked products
Output	The requested page	`LocalShopOutput`	`results` , `parsed` (what was understood, including the darija translation), `total_matches` , `note` , `search_id`

Real examples (catalogue of 16/10, 43 shops)

Each line is a real API response, with `k = 3` (the first 3 results).

Input sent to M7	What M7 understood (`parsed`)	Output (`total_matches` then results)
`{"query": "baskets blanches à Tunis pour un budget de 80 TND"}`	sneakers · white · Tunis · ≤ 80 TND	**1** — Manuella · Tunis · Basket à plateforme femme · 69.3 TND (0.880)
`{"query": "robe noire", "city": "Sousse"}`	dress · black · Sousse	**27** — Ms. Marion · black knit long dress · 92 TND (0.865); Maison ARA · Robe Bretelle Gaga · 119 TND (0.861); Hraier · black wool jerbia dress · 225 TND (0.821). *note*: "Rien trouvé à Sousse : résultats d'autres villes ou de boutiques en ligne."
`{"query": "sbadri kahla a9al men 150 alf"}` (darija)	`translated` = "baskets noir moins de 150 alf" → sneakers · black · ≤ 150 TND	**47** — Exist · Baskets · 119.99 TND (1.0); HA · Basket Femme · 129.99 TND (0.963); Carolie · Sfax · Baskets Chics · 89.9 TND (0.956)
`{"query": "سروال رجالي"}` (Arabic)	"pantalon homme" → trousers · men	**83** — NOON · tapered trousers · 39.9 TND (0.941); GET · jogger · 64.95 TND (0.926); GET · zip jogger · 71.92 TND (0.902)
`{"query": "chemise homme blanche"}`	shirt · white · men	**25** — Cravate · open-collar poplin shirt · 89 TND (0.968); mandarin-collar poplin shirt · 59.43 TND (0.954); linen-look shirt · 55.3 TND (0.906)
`{"query": "chemize blanche"}` (typo)	`corrected` = "chemise blanche"	**42** — Cravate · 89 TND (0.951); Ms. Marion · Dalya Shirt · 34.5 TND (0.929); Cravate · 55.3 TND (0.896)
`{"subcategory": "bag", "color": "camel", "max_price_tnd": 150}` (fields only)	bag · camel · ≤ 150 TND	**19** — Carolie · Sfax · leather shoulder bag · 79.5 TND (1.0); Hraier · caramel crochet tote · 99 TND (1.0); SeaSide · Sfax · Sac Medium Camel · 119.9 TND (0.984)
`{"query": "zorglub"}` (unknown word)	nothing	**0** — *note*: "Je n'ai trouvé aucun article correspondant à « zorglub » dans les boutiques locales."
`{"query": "robe", "k": 24, "offset": 24}` (page 2)	dress	**193** — the next 24 dresses, without repeating page 1
`{"city": "Tunis"}` (nothing to search)	—	**422** error: the input is refused before the model

Key points. - A **requested colour is strict**: "robe noire" only shows black dresses. If the colour exists nowhere, M7 says so in `note` and shows other colours (§5.5). - The `area` field gives **the city of the store that matches the filter**; for a chain without a filter, "Chaîne présente dans 4 villes (Tunis, Sfax, Sousse…)". - `match` (0 to 1) and `reasons` are computed only from the data and the scores.

5.1 Architecture and contract

The module follows the structure of shared modules (`backend/modules/m07_local_catalogue/` with `contract.py` , `router.py` , `service.py` , `mock.py` , `tool.py` , `tests/`), plus `agent.py` , `darija.py` , `events.py` , `pipeline/` and `evaluation/` , and the pipeline of the architecture: **parse** → **retrieve** → **filter** → **rerank**.

Input (`LocalShopInput`) — at least one of `query` , `gap` , `category` / `subcategory` or `image_embedding` :

Field	Example	Used by
`query`	"baskets blanches à Tunis pour 80 TND", "sbadri kahla"	Chat (BO8), search screens
`gap`	"white shirt"	Smart Buy (BO5), wardrobe analysis
`city`, `max_price_tnd`, `min_price_tnd`	"Tunis", 80	Explicit filters
`category`, `subcategory`, `color`, `gender`	shoes, sneakers, white, women	Explicit filters
`user_id`, `style_vector`	"usr_042", "vec_u042" (or 512 numbers)	Style profile (M5)
`image_embedding`	512 numbers	Photo of a piece (Vision M1 / Smart Buy)
`include_unavailable`, `k` (≤ 100), `offset`	false, 24, 24	Options and pagination

Output (`LocalShopOutput`): `results` has the fields of the architecture (`shop`, `area`, `item`, `price_tnd`, `match`), enriched with the id, links, photo, sizes, gender, stock and **reasons**. Plus `parsed` (filters applied, translated or corrected request), `total_matches`, `note` and `search_id` (id of the search in the log, §5.8).

5.2 FashionCLIP index (knowledge base)

FashionCLIP (`patrickjohncyh/fashion-clip`) is a neural network (CLIP architecture) trained by researchers on about 800,000 fashion photos with their descriptions. It turns **a photo into a list of 512 numbers** (a "vector") and **an English sentence into another list of 512 numbers**, so that a photo and its right description have close vectors. It is used as is (*zero-shot*), without retraining.

Why FashionCLIP: it understands **visual content** even when the description is poor; the architecture recommends it for the Vision module, so M7, M1 and M5 share the same space; it runs **locally** on CPU.

The `ml/data/shops/index/` folder holds three files:

File	Content
`image_embeddings.npy`	Array of **6,375 × 512 numbers** (float32, 13 MB), one normalised vector per photo
`ids.json`	The `product_id` of each row of the array, in the same order
`meta.json`	Model used, dimension, number of products, build date and duration

`pipeline/ingest.py` builds the index **incrementally**: a product already present (same id, hence same photo) is reused. For campaign 2, **4,129 new photos were encoded in 12 min 28 s** on CPU, and the 2,465 older ones reused. If the model changes, `meta.json` prevents mixing vectors of two models (`--full` re-encodes everything).

5.3 Understanding the request (`pipeline/parse.py`, `darija.py`)

The user's sentence is turned into criteria:

```
"Je cherche une chemise en lin blanche moins de 100 dt à la Marsa"
  → subcategory: shirt · color: white · max_price_tnd: 100 · city: La Marsa (Tunis governorate)
  → FashionCLIP prompt: "a photo of white linen shirt"
```

It recognises budgets ("moins de 80 dt", "entre 50 et 120 dinars", "80 alf"), Tunisian cities, colours, categories and attributes (lin → linen, cuir → leather, soirée → evening…).

Darija. Many requests are written in Tunisian darija, in Latin letters with digits for Arabic sounds ("a7mar", "5adhra", "9ahwi") or in Arabic script ("صباط أحمر"). A **lexicon** of about 40 entries (`darija.py`) replaces each darija word by the French word the rules already know, **before** anything else: pieces (*sabbat*, *sbadri*, *sorya*, *sarwel*, *kabbout*, *chkara*…), colours (*ahmar*, *kahla*, *bidha*, *azra9*, *dhahbi*…, with their feminine Arabic forms such as ذهبية), gender (*rjel*, *nsa*, رجالي, نسائية), occasion (*3ers* = wedding) and budget (*a9al men* = less than). An article or preposition glued to Arabic words is handled ("بالأسود"). `parsed.translated` shows how the request was read, and the app displays it ("Darija comprise : « baskets noir moins de 150 alf »"). Only words with one clear meaning are listed: "jupe" (skirt) has no entry, its darija spellings being too close to "jebba".

Gender. "homme", "femme", "women's", but also "pour mon mari" (for my husband) or "lel nsa" give the `gender` criterion, and the prompt becomes "a photo of men's white shirt".

Typos. An unknown word very close to a known one is corrected ("chemize" → "chemise"). **Rule keywords come first**: a shop that writes a typo itself does not make it "valid". A word already understood ("noires", "dorée") is never touched.

Why a rule-based parser and not an LLM: it uses **exactly the same word lists** as cleaning, it is instant, free, deterministic and testable; and it is measured (§6: 100 % of the 72 requests understood).

Why an English prompt: FashionCLIP was trained on English only (§5.12). The other languages (French, darija, Arabic) are handled by our rules, not by FashionCLIP.

5.4 Hybrid retrieval (`pipeline/retrieve.py`)

Two scores are computed for **every** product: - **Visual (FashionCLIP)**: cosine similarity between the prompt (or the photo sent) and the product photo; - **Text (BM25)**: a classic word-search algorithm on the name, category, colour, shop and city. Gender words are left out: a "men" piece recognised from its photo does not necessarily have "homme" in its name.

Why hybrid: the visual score finds badly described pieces; the text score catches precise words (a shop name, "jebba") the image does not contain. *Why exact computation:* with ~6,400 products, comparing to all of them takes a few milliseconds; a vector database only pays off at a larger scale.

5.5 Filters (`pipeline/filter.py`)

Filter	Type	Reason
Price ≤ budget, price ≥ minimum	Strict	The TND budget is a user constraint
City **or its governorate**	Strict	"Tunis" includes La Marsa; a shop without an address is not put in a random city
Category / subcategory	Strict	A request for sneakers must not return sandals

Filter	Type	Reason
Colour	**Strict**	"robe noire" = black dresses only; products of unknown colour are left out
Gender	Strict for the other gender	A "men" request drops women's pieces; unisex and unknown pieces stay, ranked lower
In stock	Strict (can be turned off)	Do not suggest what cannot be bought

Why colour became strict. In the first version, colour was only part of the score. With 7 black dresses in stock and 24 results shown, "robe noire" also showed blue dresses, and UNJA's "Robe Nejma" (a navy jebba-style dress whose colour the shop does not give). The user asked for "only this colour". The evaluation (§6) measured the effect: wrong results in the top 5 drop from **8.1 % to 1.1 %**.

When nothing matches, one criterion is relaxed at a time, and `note` says so: 1. first the **city** ("robe noire" in Sousse shows black dresses from other cities, not blue dresses from Sousse); 2. then **stock** (sold-out pieces, labelled "Rupture de stock"); 3. last the **colour**, only if it exists nowhere ("Aucun article de couleur rouge parmi les baskets : voici les autres baskets disponibles.").

Nothing understood (no type, colour or photo): only products containing the searched words are kept, otherwise the engine answers "Je n'ai trouvé aucun article correspondant à …".

5.6 Score, style, diversity, pagination and explanations (`pipeline/rerank.py`)

The `match` score (between 0 and 1) is a weighted mean of the parts that apply:

Part	Weight	Measure
Visual	0.45	FashionCLIP query ↔ photo, mapped to 0–1
Text	0.20	BM25, relative to the best candidate
Attributes	0.20	Requested subcategory, colour and gender (gender: same or unisex = 1, unknown = 0.5)
Style	0.15	User's style vector ↔ product photo

Weights are **renormalised** over the parts available.

Calibration. FashionCLIP similarities are low in absolute value: text → photo (median ≈ 0.11, 99th percentile ≈ 0.30), hence the range [0.15; 0.33]; photo → photo, range [0.40; 0.85].

Diversity (MMR) and pagination. We avoid returning 5 almost identical photos (the same model in 5 colours). MMR picks results one by one among the **2,000 best candidates**, a fixed number: its first n picks therefore do not depend on the page size. Page 2 (`offset = 24`) continues page 1 exactly, without repeating or missing a piece. Checked on "robe": all 193 dresses are shown, each exactly once, in less than 120 ms per page.

Explanations without invention. Each result has reasons built only from the data and the scores: "Baskets", "Couleur : blanc", "Modèle homme (estimé d'après la photo)", "69.3 TND (budget 80 TND)", "Boutique à Tunis (près de La Marsa)", "Ressemble visuellement à la demande", "Marque tunisienne". They feed the BO6 screen and the LLM module (M8) without letting a model invent a fact.

5.7 Gender estimated from the photo (`pipeline/attributes.py`)

Shops write the gender for only 1,834 of the 6,375 products. For the others, FashionCLIP compares the photo with sentences such as "women's shirt", "a man wearing a shirt"... (4 phrasings per gender and per type of piece, averaged). The **margin** = 100 × (similarity "women" − similarity "men"). The gender is kept only if the margin reaches **1.5** (`config.yaml`); otherwise it stays unknown.

Why a threshold: a wrong "men" would hide a piece from a women's search, while an unknown gender only ranks it a little lower.

Evaluation. Products whose name gives the gender are the test set: the photo prediction is compared with what is written.

Minimum margin	Products covered	Correct predictions
0	100 %	93.3 %
1	92.2 %	95.6 %
1.5 (used)	**88.5 %**	**96.7 %**
3	72.8 %	98.8 %

Measured on 1,828 products (`ml/data/shops/attributes_report.md`). Predictions are consistent shop by shop: Exist, a menswear brand, gets 266 "men" out of 267.

Colour from the photo: tried, not kept. The same method finds the written colour in only **47 % of cases** (57 % when grouping close shades such as blue and navy): too unreliable to fill the 15 % of missing colours. It remains a lead (§8).

5.8 Search and click log (`events.py`)

Every real search (API or assistant, not the mock) and every click is appended to `ml/data/logs/m07_events.jsonl` (not in git). One line per event, with the fields of the project's shared `Event` schema (`event_id` , `user_id` , `type` , `target_id` , `value` , `channel` , `ts`) and a `data` object: - `local_search` : the request, the filters, what was understood, the number of results, the products shown (shop, `match` , price), the note and the latency; - `product_view` (product sheet opened) and `shop_click` ("Voir sur le site de la boutique"): product, shop, `search_id` and rank, sent by the app through `POST /m07/events` .

No IP address or device data is kept; `user_id` only when given. `python -m modules.m07_local_catalogue.events` summarises it: volume, share of darija, searches without result (the **unmet demand**), and impressions, views and clicks per shop. This is the starting data of the **boutique dashboard** of module M14.

5.9 Integration in the project

- **Orchestrator tool**: `tool.py` exposes the agent as `localshop` (discovered automatically); its searches are logged with `channel = orchestrator` .
- **REST API** (`router.py` , prefix `/m07`):
- `POST /m07/search` : search (with `k` and `offset` to page);
- `POST /m07/events` : clicks from the app;
- `GET /m07/shops` : list of shops;

- `GET /m07/products/{product_id}` : one product;
- `GET /m07/images/{name}` : local photo of a product.
- **Without the data** on the machine, the service answers with `mock.py` instead of failing.
- **BO6 screen** (`mobile/lib/features/bo6_local_market/`): search, city and budget filters, understood criteria (including darija and gender), photo grid with "Voir 24 de plus" and "Tout afficher", product sheet with "Voir sur le site de la boutique" (opens the page) and "Copier le lien".
- The model is loaded at the first search (about 12 s), then **each search takes about 40 ms** (median 37 ms, 95th percentile 42 ms).

5.10 Real result examples

Request	Result
"robe noire"	27 black dresses from 12 shops (Ms. Marion, Maison ARA, Hraier…); 7 before campaign 2
"jebba"	73 traditional outfits (Hraier first: modern jebba 120 TND); 13 before
"maillot de bain"	29 swimsuits (Lyséa · Tunis first); 5 before
"collier doré"	90 gold necklaces (Gaya first, 29.9 – 34.9 TND)
"bracelet"	56 bracelets (Iley'com, Strass, Hraier)
"costume"	26 suits (Exist, GET)
"sandales à Sfax"	245 sandals (SeaSide, Carolie, Baletto · Sfax)
"escarpins rouges"	2 red heels (SeaSide · Sfax)

5.11 Fixes after testing in the app

First feedback (14/10):

Search	Before	Cause	After
"costume"	Suits then bracelets, caps, sweaters	Unknown word + list always filled	Suits only
"colier" (typo)	Random list	No typo tolerance	Necklaces
"bracelet"	1 bracelet + 4 rings	All jewellery in one category	Bracelets only
"robe blanche" + Sousse	"MAGIC · Tunis" shown	The card showed the chain's first city	`area` = city matching the filter

Second feedback (15–16/10):

Search or problem	Before	Cause	After
"chemise homme blanche"	"Chemise Femme" in the top 5	Gender ignored	5 men's shirts out of 5 (gender read from the name, else estimated from the photo)
"sabbat ahmar", "سروال رجالي"	0 results	Darija and Arabic not understood	Darija lexicon: red shoes, men's trousers
"robe noire"	Blue dresses and a blue jebba among the 24	Colour only in the score	Strict colour: black dresses only

Search or problem	Before	Cause	After
"bague argentée", "leather handbag", "سنسلة ذهبية"	Colour or type missed	Feminine forms and English words missing	Found by the test set (§6), rules fixed
195 dresses, 24 shown	No way to see the others	No pagination	`offset` + "Voir 24 de plus" / "Tout afficher" buttons
Shop link	Only copied	No URL opening in the app	"Voir sur le site de la boutique" opens the page
Slow app (≈ 2 s per call)	App server forwarding to `localhost`	Windows tries IPv6 first	App server forwards to `127.0.0.1` : 20 to 60 ms

5.12 Is FashionCLIP the best choice?

Comparison on our catalogue (campaign 1). Three fashion models were evaluated on 1,873 photos, with the shops' labels as reference (105 "colour + type" searches):

Measure	**FashionCLIP (kept)**	Marqo-FashionSigLIP	Marqo-FashionCLIP
"Colour + type" search: correct results in top 10	29.7 %	**32.5 %**	30.9 %
Type search: correct results in top 10	67.6 %	68.9 %	**69.4 %**
Type recognised on the photo	66.8 %	**72.0 %**	67.3 %
Colour recognised on the photo	47.9 %	47.5 %	**48.9 %**
CPU speed	**96 ms/photo**	234 ms/photo	236 ms/photo
Vector size	**512**	768	**512**

Decision: keep FashionCLIP for now. Recent models gain 1 to 3 points (close to measurement noise); FashionCLIP is 2.4 times faster and follows the project's 512 convention. Marqo-FashionSigLIP (768) would also require changing M1 and M5: a team decision.

Languages. FashionCLIP really understands English only (77 % correct types in the top 10 in English against 58 % in French on 12 direct searches). That is why French, darija and Arabic go through our rules. The evaluation of §6 confirms it at a larger scale: FashionCLIP alone on the raw request gets 0.03 nDCG on darija, against 0.97 for the pipeline.

5.13 The agent's LLM: Qwen 2.5 7B, local

Since 22/10, the `localshop` agent (M7 and its M14 skill) has **an LLM: Qwen 2.5 7B**, run on the machine with **Ollama** (`llm.py`). Why this model: it is free and local (no API key, the clients' requests never leave the machine), it fits in memory on a 32 GB PC without a GPU, and it is good in French and Arabic, which matters for darija.

Its role in M7: understand the requests the rules do not understand. The rules of §5.3 stay the first step, fast and exact. The LLM is called only when they find **no piece type, no colour and no photo** (`pipeline/understand.py`):

```
"haja nlabesha l b7ar fi sif"  ──►  rules: nothing understood
        │
        ▼  Qwen 2.5 7B, JSON output forced by a schema (Ollama structured outputs)
{french: "…", subcategory: <catalogue code>, color: <code>, gender, max_price_tnd, city}
        │
        ▼  checks in the code
budget written in the request? city and gender said in the request? otherwise removed
        │
        ▼  the normal search (FashionCLIP + BM25 + filters) runs on these filters
kept only if it finds pieces; note: "Demande comprise avec l'IA (qwen2.5:7b): …"
```

The LLM **never sees the products and chooses nothing**: it turns the request into filters, with a fixed vocabulary (the catalogue's codes). When it is off, slow or wrong, the answer is the rules' one, as before. The tests use a fake LLM; they never call Ollama.

Two lessons from tuning it, measured on the requests of §6.6: - with examples in the prompt, the small model **copied them** into unrelated answers ("beach outfit" for a work outfit): the prompt has no examples any more and asks for "the most likely piece" for the use (beach → swimwear, winter → coat, interview → suit); - the model **invented a gender** ("women"): a gender is kept only when the request says it.

5.14 Tunisian occasions, seasons and the usual budget (BO6 ideas 8 and 9)

Idea 8 · Outfits for Tunisian occasions and seasons (`pipeline/context.py`). One lexicon in French, English, darija and Arabic reads the occasion (mariage / 3ers / عرس, henna, khotba, Aïd, Ramadan evenings / sahria / سهرية, interview, graduation, work, evening party, beach / b7ar) and the season (été / sif / صيف, hiver / chta / شتاء). M14's hand-off summary uses the same lexicon.

Request	What the agent does
"tenue pour un mariage", "n7eb lebsa l 3ers o5ti", "عندي عرس في سوسة"	No piece named: it searches the wedding pieces (dresses, sets, suits, jumpsuits, traditional outfits, heels) and says so in the note
"robe pour un mariage"	The piece decides (dresses); the occasion adds "elegant wedding guest" to the FashionCLIP prompt
"robe d'été" / "robe pour l'hiver"	The same 193 dresses, but those whose name and type say the season come first (straps, linen, crochet / wool, knit, velvet); none is hidden

On the real catalogue these requests are understood **by the rules, in under a second**, with no LLM call (before, "an outfit for a job interview" needed ~15 s of Qwen).

Idea 9 · Budget in dinars, learned (`memory.budget_samples` , `agent._usual_budget`). The agent keeps the budgets the user gave and the price of what they bought (feedback), per type of piece. From 2 values, the median becomes their **usual budget** for that type. When a request has no budget, the pieces within it come first (`budget` part of the score), a memory note says so ("Your usual budget for sneakers: ~90 TND…") and each such piece gets the reason "Within your usual budget". A budget said in the request always wins, nothing is hidden, and `DELETE /m07/memory/{user}` forgets it. Real example: after "baskets moins de 80 dt" and "baskets blanches max 100 dt", "baskets" first shows pairs between 40 and 91 TND instead of 130 to 183 TND.

Tests: `tests/test_context_budget.py` (15 tests).

5.15 The local piece with the wardrobe: M1 + M2 (BO6 ideas 3, 4 and 10)

- **Idea 10 · "Why this local piece"**: for a known user, the top 3 results also say which pieces of **their wardrobe** they go with ("Va avec ta garde-robe : jean (bleu), baskets (blanc)…"). The wardrobe comes from M1, the compatibility from M2 (OutfitTransformer).
- **Idea 3 · Local piece → complete outfit**: `POST /m07/outfits` builds outfits from the wardrobe around a shop piece (M2 `/complete`, the piece pinned and sent with its FashionCLIP vector), and says why this piece: Tunisian brand, shop, city, price, wardrobe pieces it goes with.
- **Idea 4 · Tunisian brands in the outfit of the day**: `POST /m07/local-pieces` takes an outfit of the wardrobe (for example the one BO1 recommends) and proposes a Tunisian shop piece that completes it (shoes, a bag, a jacket…) or replaces one of its pieces, scored by M2 with the whole outfit. Example: jeans + white shirt → white leather shoes from AD Makeba, a khomsa jewellery set from Hraier, a VOG jacket.
- **Simulated until the integration**: M1 and M2 are on other branches. Until they are merged a simulation answers (a fixed wardrobe, a rule on types and colours) and the answer says so (`sources: {"m1": "mock", "m2": "mock"}`). At the integration, `real: [m1, m2]` in `config.yaml` is enough; the real path is already tested with a fake M1 / M2.
- Smart Buy (BO5) is not called directly: it is an agent, chained by the orchestrator through the `judge` next action (rule of the agent guide).

Tests: `tests/test_wardrobe.py` (10 tests).

5.16 Quality improvements (24/10)

- **Gender read on the photo: 96.7 % → 98.2 % right.** Three changes, measured on the 1,955 products whose shop writes the gender: garments that say the gender (abaya, caftan, burkini → women; qamis → men), a stricter threshold (2.0) because a wrong gender hides a piece, and a rule taken from the data: when the shops write one gender for at least 95 % of a type (dresses, skirts, heels, lingerie → women), a contrary guess is dropped (52 cases).
- **Season read on the photo** when the name says nothing ("summer …" / "winter …", same method): 86 % right on 577 products whose name says the season; 3,480 of 6,375 products now have a season, against 1,620.
- **Dressy occasions**: for a wedding, Eid or an interview, sports, beach or home pieces are left out, unless asked for ("baskets pour un mariage").
- **Freshness**: when the catalogue is older than 30 days every answer says prices and stock are to be checked; **Arabic**: notes, reasons, pieces and colours in Arabic; **3D try-on**: a `try_on` next action to BO9's agent (idea 6, BO6 side).

Tests: `tests/test_improvements.py` (11 tests).

---

6. Evaluation

6.1 Test set

`evaluation/queries.yaml` holds **72 requests** written the way users type them: 28 in French, 12 in English, 18 in darija (Latin letters) and 14 in Arabic script, with typos, budgets, cities, colours, genders and materials. For each one, **the intent is written by hand**: type, colour, gender, budget, city, and optionally material or occasion words expected in the name ("cuir", "soirée"). Example:

```
- {id: dj01, lang: darija, query: "sbadri kahla a9al men 150 alf",
   expect: {type: sneakers, color: black, max_price: 150}}
```

One command writes the report `evaluation/RESULTS.md`:

```
python -m modules.m07_local_catalogue.evaluation.run
```

6.2 Two levels of measurement

Level	Reference	Measures
Understanding	Intent written by hand (*gold*)	Share of requests whose every field (type, colour, gender, budget, city) is read correctly; "exact" = all fields right
Ranking	Grade of each product computed from the intent and its attributes (*silver*): 2 = right type and every criterion, 1 = right type but a criterion missing or unknown, 0 = wrong type, over budget, other gender or other colour	nDCG@10, P@5 (grade 2 in the first 5), MRR, wrong results in the top 5, requests without result, latency

6.3 Results (catalogue of 16/10, 6,375 products)

Understanding: 100 % of the 72 requests are read exactly (every field right), in all four languages. Without the darija lexicon: 56.9 % (0 % in darija, 7.1 % in Arabic).

Ranking — comparison of variants (ablation):

Variant	nDCG@10	P@5	MRR	Wrong in top 5	No result
BM25 alone, raw request	0.329	27.2 %	0.331	41.4 %	38.9 %
FashionCLIP alone, raw request	0.281	23.1 %	0.346	66.7 %	0 %
Pipeline without FashionCLIP	0.935	84.2 %	0.928	0.8 %	0 %
Pipeline, visual score only	0.917	82.5 %	0.903	1.1 %	0 %
Pipeline without darija lexicon	0.521	43.6 %	0.507	6.0 %	40.3 %
Pipeline without gender	0.832	74.4 %	0.845	12.8 %	0 %
Pipeline, soft colour (before)	0.906	82.2 %	0.912	8.1 %	0 %
Full pipeline	**0.934**	**84.4 %**	**0.938**	**1.1 %**	**0 %**

Per language (nDCG@10): French 0.908, English 0.905, darija 0.974, Arabic 0.962. Latency: median 37 ms, 95th percentile 42 ms.

6.4 What these figures show

- **Understanding the request is what matters most.** Word search (BM25) or FashionCLIP alone on the raw sentence cap at 0.33 nDCG; the pipeline reaches 0.93.
- **The darija lexicon is essential**: without it, nDCG drops to 0.52 and 40 % of requests return nothing.
- **Gender and strict colour remove visible errors**: without gender, 12.8 % wrong in the top 5; with soft colour, 8.1 %; full pipeline: 1.1 %.
- **The test set found real bugs.** On the first run, 4 requests were misread ("argentée", "handbag", نسائية, ذهبية), then "colier dorée" after adding a shop that writes "colier" itself. The rules were fixed, and a test protects each case.

6.5 Limits of the evaluation

- **Silver grades use the same attributes as the filters** (type, price, colour, gender, city). They favour the pipeline on those criteria; they mainly show what understanding brings against raw methods, and how what passes the filters is ranked.
- **They do not look at the photos.** That is why "without FashionCLIP" scores almost the same as the full pipeline (0.935 against 0.934): the visual contribution (resemblance, style) cannot be measured with these grades. Only **human grades** can settle it.
- **The 100 % understanding is optimistic**: the bugs found by the test set were fixed on that same set. Requests written by other people would give an unbiased figure.
- 63 % of genders are estimated from the photo (96.7 % correct, §5.7).

Human grades. `ml/data/shops/eval/judgments_todo.csv` lists the 1,533 products shown by the pipeline and the two raw methods (top 10 of each). Grading about twenty requests is enough (`human_rel` = 0, 1 or 2, looking at the photo and the link): save the file as `judgments.csv` and run again; `RESULTS.md` then adds human nDCG@10 and P@5, and the agreement with the automatic grades (Cohen's kappa).

6.6 What the LLM adds: 16 hard requests

On the 72 requests of §6.1 the rules already understand everything, so the LLM is never called. To measure it, we wrote **16 requests the rules do not understand** (French, English, darija, Arabic: "haja nlabesha l b7ar fi sif", "an outfit for a job interview", "عندي عرس في سوسة نحب حاجة باهية"...), with their intent written by hand (`evaluation/queries_llm.yaml` , script `llm_eval.py`).

	Type understood and right	Budget / city right	Pieces found	Median time (CPU)
Rules alone	0/16	15/16	0/16	–
Rules + Qwen 2.5 7B	**11/16**	16/16	14/16	14.8 s
Rules + Qwen 2.5 3B	12/16	16/16	16/16	7.6 s

- The LLM takes these requests from **0 to 11 out of 16** understood; no budget or city is invented (16/16).
- **7B and 3B are even** on this small set (11 vs 12 is not a real difference). The 7B rewrites better ("costume d'affaires" for an interview, "sac de femme" for a gift for a mother, where the 3B writes "robe de travail" and "robe de fête"), but it is **twice as slow** on a CPU. The model is changed in

`config.yaml → llm.model`.

- The 7B's misses: the beach read as "shirt", winter shoes read as "shirt", a warm outfit read as "evening dress", the hammam not understood.
- **Limit**: these 16 requests and their intent were written by the same team as the system; a set written by other people would give a fairer figure. The response time (≈ 15 s) comes from the lack of a GPU; that is why the LLM is only the last resort.

---

7. How to test the module

Backend commands run from `backend/`, app commands from `mobile/`.

7.1 Automated tests (no model, no internet)

```
cd backend
pip install -r requirements-dev.txt
pytest modules/m07_local_catalogue -q     # expected: 128 passed
pytest -q                                 # whole backend: 153 passed
cd ../mobile && flutter test              # BO6 screen: 5 tests
```

These tests use a tiny catalogue and a fake encoder: they need neither `torch`, nor the model, nor the data.

File	What is checked
`test_search.py`	"baskets blanches à Tunis 80 TND" example, governorate, strict colour and relaxation order, pagination without repeats, typos, jewellery and suits, unknown words, sold-out pieces
`test_darija_gender.py`	Darija in Latin letters and Arabic, budget in "alf", gender in the request and in the name, gender filter and reasons, gender from the photo
`test_evaluation.py`	Relevance grades, nDCG / P@5 / MRR, understanding per field, full run on a tiny catalogue
`test_events.py`	Search and click log, `search_id`, `/m07/events` route, summary for M14
`test_router.py`	`/m07/*` routes, mock fallback without data, protected photos, style vector asked to M5
`test_scraper_normalize.py`	Categories (FR/EN/Tunisian/Arabic), typographic apostrophes, "haut de gamme", two-level non-fashion filter, foreign labels
`test_clean.py`, `test_ingest.py`, `test_contract.py`, `test_mock.py`	Cleaning, incremental index, contract, mock
`mobile/test/bo6_local_market_test.dart`	BO6 screen: search, understood criteria, TND prices, "Voir plus" button

7.2 Requirements for a real search

```
pip install -r modules/m07_local_catalogue/requirements.txt   # torch, transformers...
```

The data is in `ml/data/shops/` (fetched with `git pull`). To rebuild it:

```
python -m modules.m07_local_catalogue.scraper.run          # 1. scraping, photos (~1 h)
python -m modules.m07_local_catalogue.scraper.clean        # 2. cleaning, URLs (~55 min)
python -m modules.m07_local_catalogue.pipeline.ingest      # 3. index (incremental)
python -m modules.m07_local_catalogue.pipeline.attributes  # 4. gender from the photo
python -m modules.m07_local_catalogue.evaluation.run       # 5. evaluation (~3 min)
```

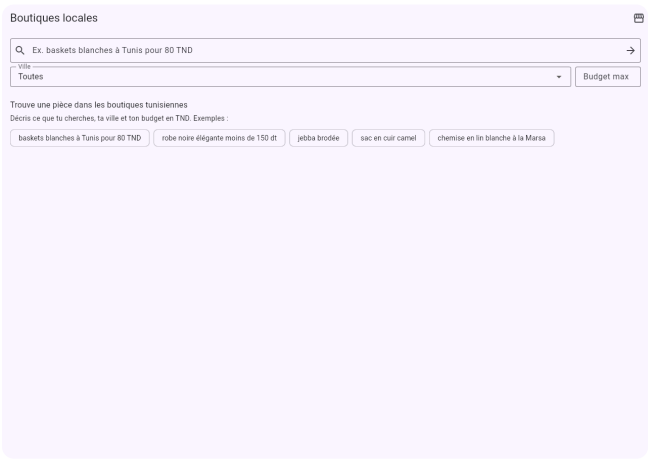
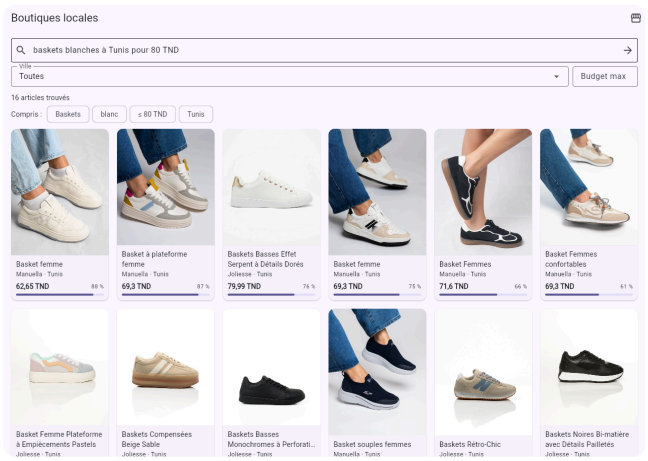
7.3 Testing through the API (Swagger)

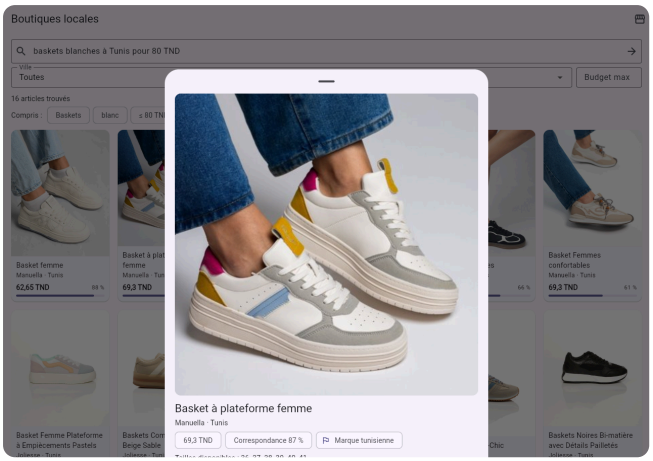
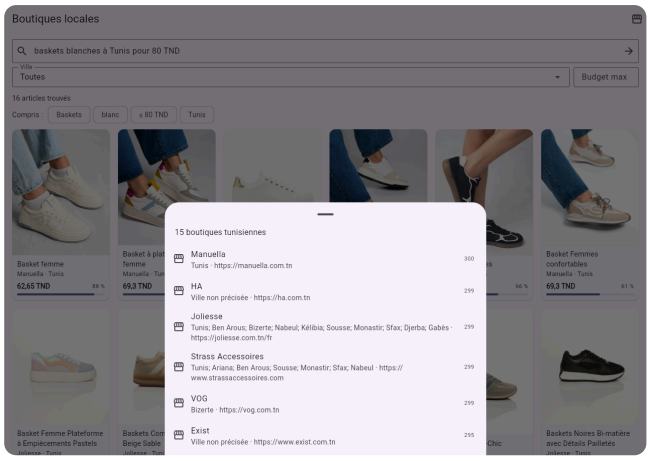
```
cd backend
uvicorn api.main:app --host 127.0.0.1 --port 8010
```

Open **http://127.0.0.1:8010/docs**, choose `POST /m07/search`, **Try it out**, paste a JSON body (see the table of §7.4), **Execute**.

7.4 Testing in the app (Flutter front end)

The BO6 "Boutiques locales" screen (`mobile/lib/features/bo6_local_market/`) calls M7 through the API, as the final app will.

Home	Results
	
Product sheet	**Shops**

Home	Results
	

Screenshots of the first version of the screen (14/10).

1. **Start the API** (terminal 1): `cd backend` then `uvicorn api.main:app --host 127.0.0.1 --port 8010`
2. **Build the web app** (once, then after each Dart code change): `powershell cd mobile flutter pub get flutter build web --dart-define=API_URL=http://localhost:8090`
3. **Serve the app and the API from one address** (terminal 2): `python tool/serve_app.py`. *Why:* the browser blocks calls to another port (CORS rule). `serve_app.py` serves the app and forwards `/m07/...` calls to the API: any browser works, with no special flag.
4. Open **http://localhost:8090/#/bo6_local_market**. On a phone: `flutter run --dart-define=API_URL=http://<PC-IP>:8010` (API started with `--host 0.0.0.0`).

Action	Expected result
"baskets blanches à Tunis pour 80 TND"	1 white sneaker (Manuella); "Compris : Baskets · Blanc · ≤ 80 TND · Tunis"
"robe noire"	27 dresses, **all black**
"robe"	193 pieces, 24 shown, then "Voir 24 de plus" and "Tout afficher (169 restants)"
"sbadri kahla a9al men 150 alf"	"Darija comprise : « baskets noir moins de 150 alf »", 47 black sneakers ≤ 150 TND
"chemise homme blanche"	"Homme" pill, white men's shirts (Cravate)
"chemize" (with the typo)	"Recherche corrigée : « chemise »"
Tap a piece	Photo, price, match, sizes, reasons (including "Modèle homme (estimé d'après la photo)"), "Voir sur le site de la boutique" and "Copier le lien" buttons
Shop icon (top right)	The 43 shops with their cities and website
`{"city": "Tunis"}` (API)	**422** error (nothing to search)

The app starts on the BO2 screen (shared skeleton, `mobile/lib/app/router.dart`): open the address with `#/bo6_local_market`.

7.5 Testing directly in Python

```
import asyncio
from modules.m07_local_catalogue.agent import LocalShopAgent

agent = LocalShopAgent()
out = asyncio.run(agent({"query": "sbadri kahla a9al men 150 alf", "k": 5}))
print(out["parsed"])              # understood criteria, including "translated"
for r in out["results"]:
    print(r["match"], r["shop"], r["area"], r["item"], r["price_tnd"], r["reasons"])
```

7.6 Checking the data

- Open `ml/data/shops/local_catalogue_clean.csv` in Excel (UTF-8 encoding).
- Read `cleaning_report.md` (cleaning) and `attributes_report.md` (gender from the photo).
- Filter the `quality_flags` column to see prices to check.

---

8. Limits and next steps

Limit	Next step
57 % of products without a city (online shops without an address)	Contact the shops; a "delivers everywhere" notion
Products without a colour are left out of a colour search (e.g. a black MAGIC dress)	More reliable colour from the photo: dominant colour of the cut-out garment (M1), measured with the test set
Occasion and material only read from the text ("robe de soirée" also returns day dresses)	Zero-shot "evening", "leather", "linen"... attributes on the photos, evaluated
Outlier prices flagged but kept (Hraier bags at 14,900 TND)	Manual review of `price_very_high`
Shops not covered: ZEN and Souk El Kahina (robots.txt), Mabrouk and HKEYA (technical refusals), Kastelo, Barsha...	Ask for a partner file (CSV) or the shops' agreement, as the architecture plans; dedicated HTML parsers
Lingerie very present (606 products, 4 shops)	Per-category cap when balancing
Ranking evaluated without human grades; understanding measured on the set used to fix it	Fill `judgments.csv`; 50 requests written by other people
Score weights set by hand	Learn them on human grades, then on the clicks of the log (*learning to rank*)
Words missing from the lexicon not understood (the engine says so)	Multilingual text signal (bge-m3, e5) on product names; M8's LLM for unusual sentences
Photos in git (≈ 330 MB)	Move `ml/data/shops/images/` to DVC or Git LFS if the repository gets too heavy
Data frozen between two scrapes	Scheduled re-run (M4) with `--merge`; 7-day URL cache

9. Appendix: module files

```
backend/modules/m07_local_catalogue/
├── agent.py            M7 agent: parse → retrieve → filter → rerank, relaxation, pagination
├── contract.py         input / output (Pydantic), ClickEvent
├── darija.py           darija lexicon (Latin letters and Arabic) → French
├── events.py           search and click log + summary for M14
├── taxonomy.py         vocabulary: categories, jewellery, colours, genders, cities, governorates
├── config.yaml         model, weights, calibration, gender threshold, log, data location
├── paths.py, clients.py, router.py, service.py, mock.py, tool.py
├── pipeline/
│   ├── parse.py        understanding (darija, gender, typos) + English prompt
│   ├── retrieve.py     FashionCLIP + BM25
│   ├── filter.py       price, city, category, colour, gender, stock
│   ├── rerank.py       score, style, diversity (MMR), reasons
│   ├── encoder.py      FashionCLIP run locally
│   ├── catalogue.py    loads the knowledge base + index + vocabulary
│   ├── ingest.py       incremental index build (offline)
│   └── attributes.py   gender from the photo + evaluation (offline)
├── scraper/
│   ├── shops.py        43 shops kept, shops left out (with reasons)
│   ├── sources.py      one adapter per platform
│   ├── client.py       polite HTTP client (robots.txt, delays, no cookies)
│   ├── normalize.py    categories, colours, sizes, labels, non-fashion, children
│   ├── images.py       photo download
│   ├── run.py          scraping run (--shops, --merge, --delay)
│   └── clean.py        cleaning + URL checks (7-day cache)
├── evaluation/
│   ├── queries.yaml    72 requests with their hand-written intent
│   ├── run.py          understanding, ranking, ablation, human grades
│   └── RESULTS.md      generated results
└── tests/              128 tests
ml/data/shops/          module data (in git, branch BO6/local_catalogue)
├── local_catalogue.csv / .xlsx   raw scraping data (6,760 products)
├── local_catalogue_clean.csv     cleaned knowledge base (6,375 products)
├── cleaning_report.md, attributes_report.md
├── images/ (6,760 photos)  index/ (FashionCLIP)  url_checks.json
└── eval/judgments_todo.csv       products to grade by hand
ml/data/logs/m07_events.jsonl     search log (not in git)
mobile/lib/features/bo6_local_market/   BO6 "Boutiques locales" screen (Flutter)
mobile/test/bo6_local_market_test.dart  screen tests (5)
mobile/tool/serve_app.py                web app + API from one address
rapport/build_pdf.py                    builds the PDFs of this report
```